Introduction to Software Engineering

Project Proposal & Plan



Software Engineering Department
Faculty of Information and Technology
University of Science

Table of Contents

Objectives **1**

1 Member Contribution Assessment **2**

2 Vision **3**

2.1 Preliminary Problem Statement 3

2.2 Product Position Statement 3

2.3 Stakeholders and Users 3

2.4 Scope and Key Features 4

3 Proposed Solution **5**

3.1 Preliminary Architecture 5

3.2 Technology Stack 5

3.3 Hardware and Operating Environment 5

4 Project Plan **6**

4.1 Process and Phase Gates 6

4.2 Work Breakdown Structure (WBS) 6

4.3 Estimation 6

4.4 Schedule 7

4.5 Roles and Responsibilities 7

4.6 Human Resources and Cost Estimate 7

5 Risk Management **8**

6 Tools & Collaboration Setup **9**

7 AI Usage Log **11**

Project Proposal & Plan

Objectives

This document focus on the following topics:

- ✔ Completing the Project Proposal & Plan document with the following sections:
  - Vision
  - Proposed Solution
  - Project Plan
  - Risk Management
  - Tools & Collaboration Setup
- ✔ Including an AI usage log for this phase
- ✔ Understanding the Project Proposal & Plan document.

1 Member Contribution Assessment

ID	Name	Role(s)	Contribution (%)	Evidence
24127326	Đoàn Quốc Bảo	Team lead / PM	20	Link
24127025	Lê Minh Đức	Business analyst	20	
24127003	Vũ Trần Minh Hiếu	Designer	20	
24127173	Nguyễn Tuấn Hùng	Developer	20	
24127008	Trương Nhật Phát	Tester	20	

2 Vision

2.1 Preliminary Problem Statement

Background and Problem Description:

HCMUS students and staff use university portals and other platforms for academic and administrative tasks. However, these services are spread across different systems and may not fully meet users' needs. Students may need additional tools to calculate their GPA or plan their timetables, while lecturers and administrative staff need convenient ways to manage grades, tuition records, and document requests.

Portal67 is a web application that aims to bring these functions together in one place. It will provide a consistent interface for students, lecturers, and administrative staff to complete common academic and administrative tasks.

Main Functions:

Students can browse available courses and specialized topics, view seat availability and class schedules, and register for or cancel courses. The system will check basic rules, such as seat limits and timetable conflicts, using sample data.

Students can also view tuition fees, payment records, outstanding balances, and payment status. Administrative staff can update simulated payment records, but the system will not process real payments.

For academic results, lecturers can enter grades, while students can view their results and estimated semester and cumulative GPAs. The GPA calculations will follow predefined rules.

The system will also display students' weekly class timetables and examination schedules, helping them identify overlapping sessions. For document requests, students can submit requests and track their status. Administrative staff can approve or reject requests and provide comments.

Operating Environment and Technical Constraints:

Portal67 will be a web application accessible through modern web browsers on desktop and laptop computers. It will use a client-server architecture, with a frontend for user interaction, a backend for processing requests, and a database for storing information.

The planned technology stack is:

+ Frontend: Svelte and Tailwind CSS for a responsive and modular user interface.
+ Backend: NestJS or Go for REST APIs and server-side business logic.
+ Database and caching: PostgreSQL for relational data and Redis for caching and session management.
+ Hosting and deployment: Docker for containerization, Nginx for reverse proxying and load balancing, and Kubernetes for container orchestration and scaling if needed.

+ Testing and CI: GitHub Actions for automated workflows and Jest for testing.

The project has a three-week development period (Weeks 8–10). Therefore, the team will prioritize the five must-have features and choose implementation approaches that fit the available time.

The system will use sample data and will not connect to official HCMUS systems. It will not process real payments or issue legally valid documents. Basic input validation, error handling, and access restrictions will be implemented, but production-level security and availability are not guaranteed. Advanced deployment and scaling features will be used only if time and resources permit.

Expected Outcome:

The project aims to deliver a working prototype that combines common academic and administrative tasks in one web application. Its success will be evaluated based on whether the main features work correctly, are easy to use, and meet the course requirements. Portal67 is a course project prototype, not a confirmed replacement for the official HCMUS Portal.

2.2 Product Position Statement

Element	Content
For	HCMUS students, teachers, and administrative workers
Who	Need a fast, simple, feature-unified portal system.
The (product name)	Portal67
That	Is a modern, high-performance web application providing seamless course registration, timetable planning, GPA calculation, and document request & approval workflows
Unlike	The current HCMUS (or faculty) portal system and separate tools that only address individual needs.
Our product	Integrates many academic and administrative features into one application, reducing the need to switch between separate systems and tools.

2.3 Stakeholders and Users

#	Stakeholder / user	Description	Main interests
1	Students	Use the system for course registration, tuition management, academic results, timetable planning, and document requests.	Convenient and fast navigation, reliable course registration and GPA calculation, clear request status tracking.
2	Teachers	Manage course content and students' scores.	Simple grade entry, efficient workflow.
3	Admins	Responsible for tuition records and document requests.	Efficient record/request management, fast and convenient to query from database, approve or provide documents.

2.4 Scope and Key Features

ID	Feature	Description	Priority (M/S/C/W)
F01	Course & Specialized topic registration	Browse offered sections and special topics; see seats and schedules; submit/cancel registration. Validate basic seat limits and time conflicts using seeded records.	Must
F02	Tuition management	View semester charges, payment records, balances, and payment status. Staff can update simulated payment records; no money is transferred.	Must
F03	Academic result management	Lecturers enter demo grades; students view course results and an estimated semester/cumulative GPA computed from configurable sample rules.	Must

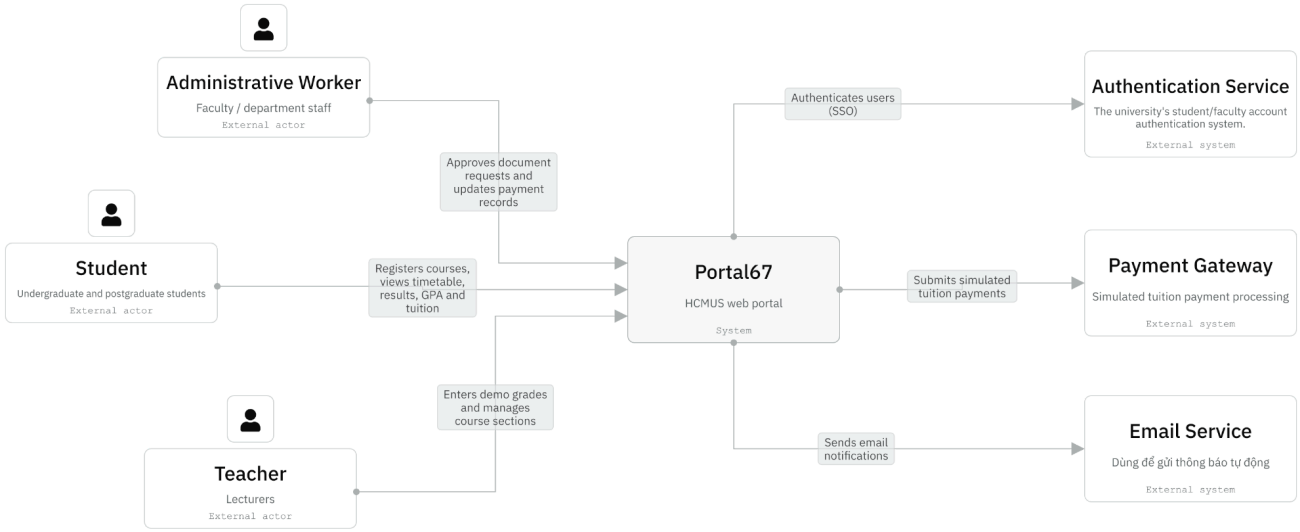
F04	Course & Exam scheduling	Show a student's weekly course timetable and exam list; display overlapping sessions and registration conflicts.	Must
F05	Resource Management	Longer-term room/equipment availability and booking for university staff. Deferred beyond the MVP unless core work finishes early.	Could
F06	Document Request & Approval	Students submit a simple document request; staff approve/reject with comments; students view status and history. No legally valid document is issued.	Must
F07	Role-based access control	Students, lecturers, and administrative staff should have different permissions. For example, students can view their own grades, while lecturers can enter grades for their assigned courses.	Must
F08	Basic input validation and error handling	Prevent invalid registrations, incomplete requests, and invalid grade or payment-record updates; show understandable error messages.	Must

Out of scope:

No features are out of scope.

3 Proposed Solution

3.1 Preliminary Architecture



3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	Svelte, TailwindCSS	High performance, modular components, responsive UI.
Backend	NestJS / Golang	Scalable microservices/REST APIs, strong concurrency handling for peak traffic.
Database	PostgreSQL + Redis	PostgreSQL for reliable relational data; Redis for fast caching/session management.
Hosting / deployment	Docker, Nginx, Kubernetes	Containerization, load balancing, and automated scaling during peak loads.
Testing & CI	GitHub Actions, Jest	Continuous integration, automated linting, and testing pipelines.

3.3 Hardware and Operating Environment

The project is deployed as a website and expected to be able to deploy on any cloud platform

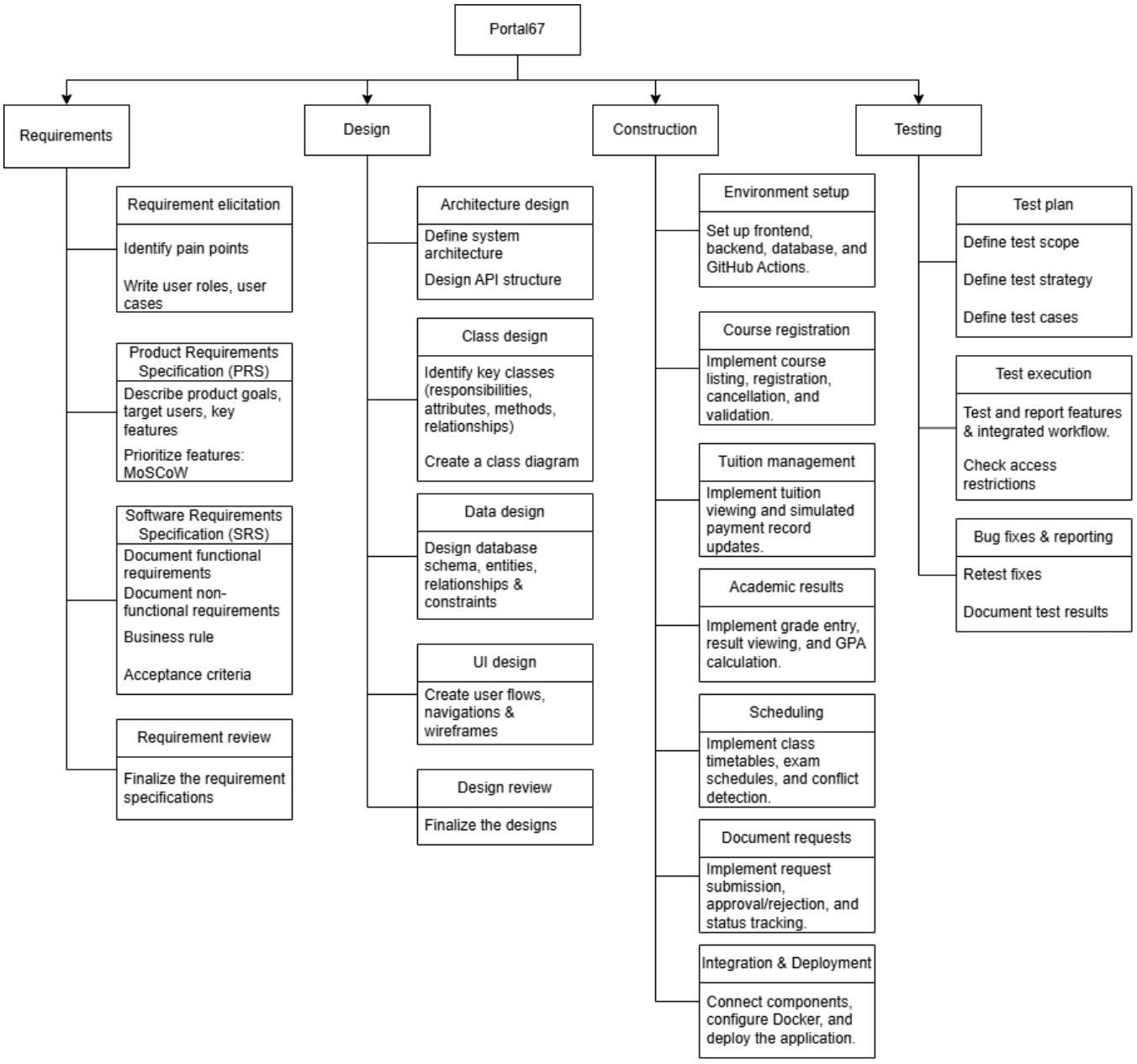
4 Project Plan

4.1 Process and Phase Gates

The project follows the waterfall model. Each phase ends with a phase gate (a PA submission). After a PA is submitted it becomes a baseline; later changes must be updated in the document and listed in the next PA.

PA	Phase	Due	Exit criteria (fill in)
PA1	Initiation & planning	End of week 3	Project idea, objectives, scope, team responsibilities, WBS, effort estimates, and project schedule are documented and reviewed.
PA2	Requirements	End of week 5	PRS and SRS are completed and reviewed. Functional and non-functional requirements, user roles, business rules, and acceptance criteria are documented.
PA3	Design	End of week 7	System architecture, database schema, class diagram, API design, and UI wireframes are completed and reviewed against the SRS.
PA4	Test planning	End of week 8	Test plan, test cases, test data, test environment, and pass/fail criteria are documented and ready for use during implementation.
PA5	Implementation , testing & deployment	End of week 10	All Must-have features are implemented, integration and testing are completed, critical defects are resolved or documented, and the application is deployed and ready for demonstration.
Fin al	Presentation & demo	Week 11	The team demonstrates the main workflows, presents the final system and project results, and submits the required final documentation.

4.2 Work Breakdown Structure (WBS)


Portal67
Requirements
Design
Construction
Testing
Requirement elicitation
Identify pain points
Write user roles, user cases
Product Requirements Specification (PRS)
Describe product goals, target users, key features
Prioritize features: MoSCoW
Software Requirements Specification (SRS)
Document functional requirements
Document non-functional requirements
Business rule
Acceptance criteria
Requirement review
Finalize the requirement specifications
Architecture design
Define system architecture
Design API structure
Class design
Identify key classes (responsibilities, attributes, methods, relationships)
Create a class diagram
Data design
Design database schema, entities, relationships & constraints
UI design
Create user flows, navigations & wireframes
Design review
Finalize the designs
Environment setup
Set up frontend, backend, database, and GitHub Actions.
Course registration
Implement course listing, registration, cancellation, and validation.
Tuition management
Implement tuition viewing and simulated payment record updates.
Academic results
Implement grade entry, result viewing, and GPA calculation.
Scheduling
Implement class timetables, exam schedules, and conflict detection.
Document requests
Implement request submission, approval/rejection, and status tracking.
Integration & Deployment
Connect components, configure Docker, and deploy the application.
Test plan
Define test scope
Define test strategy
Define test cases
Test execution
Test and report features & integrated workflow.
Check access restrictions
Bug fixes & reporting
Retest fixes
Document test results

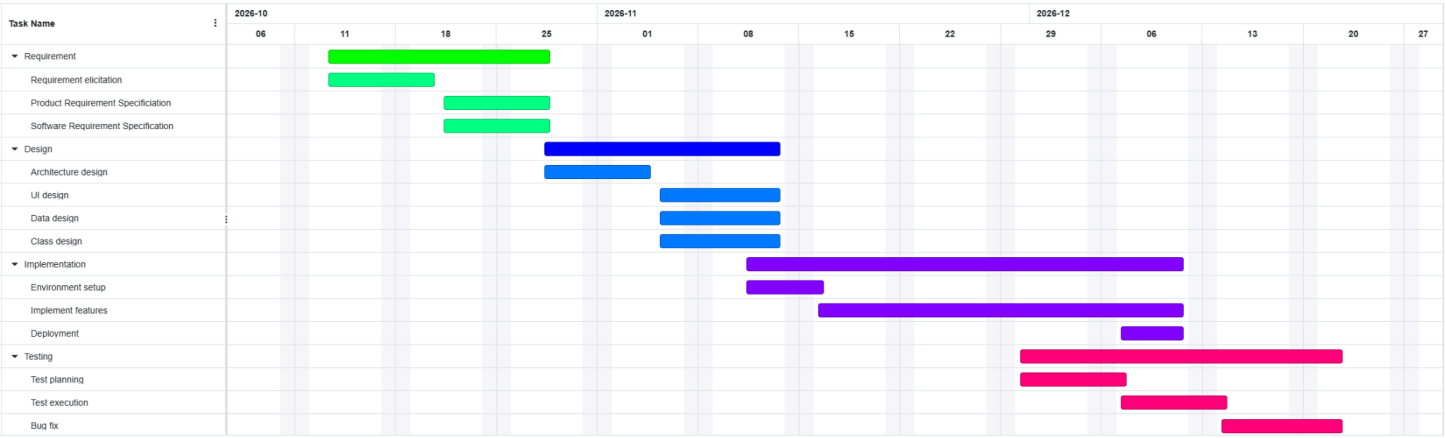
4.3 Estimation

For simple viewing, some tasks are merged.

WBS ID	Work package	Owner	Effort (person-hours)	Method / assumptions
1.1.1	Identify pain points and needs	BA	3	Analogy
1.1.2	Define roles and use cases	BA	2	Analogy
1.2	Prepare PRS: goals, users, features, priorities	BA	4	Analogy
1.3	Write the SRS	BA	5	Analogy
1.4	Finalize the requirement specifications (review)	Team lead	2	Analogy
2.1.1	Define system architecture	Designer	3	Analogy
2.1.2	Define API structure	Designer	3	Analogy
2.2	Create a class diagram	Designer	3	Analogy
2.3	Database design	Designer	3	Analogy
2.4	Create user flows, navigation, wireframes	Designer	3	Analogy
2.5	Review designs (against requirements)	Team lead	2	Analogy
3.1	Environmental setup	Developer	4	Analogy
3.2	Implement course registration	Developer	12	Analogy
3.3	Implement tuition management	Developer	8	Analogy
3.4	Implement academic results	Developer	10	Analogy
3.5	Implement scheduling	Developer	8	Analogy
3.6	Implement document requests	Developer	8	Analogy
3.7	Integration & deployment	Developer	10	Analogy
4.1	Define test scope, strategy, & test cases	Tester	5	Analogy
4.2	Test features & integrated workflows	Tester	12	Analogy
4.3	Retest fixes and document test results	Tester	6	Analogy

4.4 Schedule

For simple viewing, some tasks are merged.



4.5 Roles and Responsibilities

Role	Main	Backup	Responsibilities
Team lead / Project manager	Bảo	Đức	Plan, assign tasks, weekly status report, contact point
Business analyst	Đức	Phát	Elicit and document requirements, organise reviews
Designer / Architect	Hiếu	Bảo	Architecture, class, data and UI design
Developers	Hùng	Hiếu	Implement, unit test, review pull requests
Tester	Phát	Hùng	Test plan, test cases, system testing, test report

4.6 Human Resources and Cost Estimate

Assuming an average salary of 12 million VND/month for a junior developer (40 hours/week), each member is expected to contribute on average at least 6 hours/week in the project for 10 weeks. (Some weeks may be busier than the rest). The expected cost of labor will be at least 22.5 million VND.

Hosting and tool cost are likely to be 0 for the demo.

5 Risk Management

ID	Risk	Type	Probability	Loss	RE	Mitigation / contingency	Owner
R1	Key member unavailable during PA5	People	30%	5	1.5	Mitigation: Frequently check up on the members (meetings). Contingency: The backup member will continue tasks.	All members
R2	Frontend & Backend integration problems	Technology	50%	4	2	Mitigation: Define API contracts early; immediately test after implementing each feature. Contingency: Fallback to the most current working commit, write detailed technical documentation and have a group discussion.	Dev
R3	Decisions cannot be made (< 50% votes)	Organisational	30%	2	0.4	Mitigation: A quick survey for the members to find common ground. Contingency: The final decision is made by the team leader.	Team lead
R4	Requirement & Scope change	Requirements	30%	2	0.6	Mitigation: Debate and finalize the requirement & scope quickly. Contingency: Quickly find overlapping points between the old and new requirements/scopes and plans accordingly	BA
R5	Overwhelming tasks	Estimation	50%	2	1	Mitigation: Discuss past project experiences for better estimation. Contingency: Some tasks are shared to other members.	Team lead

6 Tools & Collaboration Setup

One of the main goals of the project is to practise teamwork for professional software development. The use of tools is monitored and graded (teamwork component). The team must use:

- ✔ Moodle: announcements and PA submissions.
- ✔ Communication: Discord
- ✔ Task board: GitHub Projects (link).
- ✔ Git repository: GitHub (Link)
  - main branch is protected; changes are merged through pull requests with at least one review
  - README (description, setup), .gitignore; never commit secrets (.env, API keys, passwords)
  - commit messages follow the Conventional Commits style (feat:, fix:, docs:, test:, …)
- ✔ Weekly meetings are determined by the team lead, and must happen before the weekly deadline (updated from PA0)
- ✔ Weekly deadline for weekly status report: 23h59 Wednesday (updated from PA0).

Repository structure:

Folder	Content
/src	Source code
/docs/management	Project plan; weekly status reports in /docs/management/weekly
/docs/requirements	Vision, SRS, use cases, mockups
/docs/design	Architecture (C4), ADRs, UML models, UI design
/docs/test	Test plan, test cases, test reports
/docs/ai-usage	AI usage logs
/pa/PA1 … /pa/PA5	One sub-folder per PA submission (PDF + source files)

7 AI Usage Log

No AI tools were used in this document.